National University of Computer and Emerging Sciences
Islamabad Operating System Lab Spring 2025

Lab Manual: Threads in C (Pthreads)

---

1. Thread States: Joinable vs Detached

Joinable Thread

- Default behavior.
- Allows the parent thread to wait for it and get a return value using `pthread_join()`.

Code:

```
pthread_t tid;

pthread_create(&tid, NULL, function, NULL);

pthread_join(tid, &result);
```

Detached Thread

- Runs independently.
- Cannot be joined or return a result to the parent thread.

Code:

```
pthread_t tid;

pthread_create(&tid, NULL, function, NULL);

pthread_detach(tid);
```

---

2. Important Functions

`pthread_self()`

Returns the ID of the current thread.

Code:

```
pthread_t id = pthread_self();

printf("Thread ID: %lu\n", id);
```

`pthread_exit()`

Used to exit from a thread and return a value.

Code:

```
pthread_t id = pthread_self();

printf("Thread ID: %lu\n", id);
```

---

3. Creating and Joining Threads

Code:

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>



void* thread1(void* arg) {

    printf("Thread 1 exiting\n");

    char* msg = "Success";

    pthread_exit((void*) msg);

}



void* thread2(void* arg) {

    printf("Thread 2 exiting\n");

    pthread_exit((void*) 2);

}



int main() {
```

```
    pthread_t tid1, tid2;

    void *ret1, *ret2;



    pthread_create(&tid1, NULL, thread1, NULL);

    pthread_create(&tid2, NULL, thread2, NULL);



    pthread_join(tid1, &ret1);

    pthread_join(tid2, &ret2);



    printf("Thread 1 returned: %s\n", (char*)ret1);

    printf("Thread 2 returned: %ld\n", (long)ret2);

    return 0;

}
```

Expected Output:

```
Thread 1 exiting

Thread 2 exiting

Thread 1 returned: Success

Thread 2 returned: 2
```

---

4. Detaching Threads

Why Detach?

- Use when we don’t need the return value of a thread.

Code:

```c
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <unistd.h>



void* thread_function(void* arg) {

    sleep(2);

    printf("Detached thread says: %s\n", (char*)arg);

    pthread_exit(NULL);

}



int main() {

    pthread_t tid;
```

```
    char msg[] = "I'm free!";



    pthread_create(&tid, NULL, thread_function, (void*)msg);

    pthread_detach(tid);



    sleep(3);

    printf("Main thread is done.\n");

    return 0;

}
```

Expected Output:

```
Detached thread says: I'm free!

Main thread is done.
```

---

5. Using Attributes to Create Detached Thread

Steps:

1. Initialize attribute with `pthread_attr_init()`
2. Set detached state using `pthread_attr_setdetachstate()`

3. Create thread with the attribute
4. Destroy the attribute using `pthread_attr_destroy()`

Code:

```c
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <unistd.h>



void* thread_function(void* arg) {

    printf("Detached thread running...\n");

    sleep(2);

    printf("Detached thread done!\n");

    pthread_exit(NULL);

}



int main() {

    pthread_t tid;

    pthread_attr_t attr;
```

```
    pthread_attr_init(&attr);

    pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);



    pthread_create(&tid, &attr, thread_function, NULL);

    pthread_attr_destroy(&attr);



    printf("Main thread won't wait. Bye!\n");

    sleep(3);

    return 0;

}
```

Expected Output:

```
Detached thread running...

Main thread won't wait. Bye!

Detached thread done!
```

---

6. Thread Cancellation

Code:

```c
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <unistd.h>



pthread_t tid;



void* thread_function(void* arg) {

    for (int i = 0; i < 10; i++) {

        printf("Thread running... %d\n", i);

        sleep(1);

    }

    pthread_exit(NULL);

}



int main() {

    pthread_create(&tid, NULL, thread_function, NULL);

    sleep(3); // let thread run for a bit



    pthread_cancel(tid); // cancel thread

    printf("Thread cancelled by main.\n");
```

```
    return 0;

}
```

Expected Output:

```
Thread running... 0

Thread running... 1

Thread running... 2

Thread cancelled by main.
```

---

7. Thread Cancel States and Types

Cancel State

- `PTHREAD_CANCEL_ENABLE` – allows cancellation
- `PTHREAD_CANCEL_DISABLE` – prevents cancellation

Code:

pthread_setcancelstate(PTHREAD_CANCEL_DISABLE, NULL);

Cancel Type

- `PTHREAD_CANCEL_ASYNCHRONOUS` – cancels immediately

- `PTHREAD_CANCEL_DEFERRED` – cancels at defined points

Code:

```
pthread_setcanceltype(PTHREAD_CANCEL_DEFERRED, NULL);
```

---

8. Summary of Attribute Functions

Function	Description
`pthread_attr_init()`	Initializes a thread attribute object
`pthread_attr_setdetachstate()`	Sets the detach state (joinable/detached)
`pthread_attr_destroy()`	Frees resources used by the attribute

THE END
